Parcial: Mi Tienda — App móvil con base de datos local y en la nube

En 90 minutos construirás una app de inventario para una tienda de barrio: el tendero inicia sesión, registra productos aunque no tenga internet y los respalda en la nube cuando vuelve la conexión. Al final subes el código a GitHub con capturas de pantalla.

Instrucciones generales

Duración	90 minutos, contados desde que el docente da la señal de inicio
Modalidad	Individual
Tecnología	Expo (SDK del curso) con TypeScript, Expo Router, SQLite con Drizzle ORM y Supabase
Entrega	Enlace al repositorio de GitHub, enviado por el canal que indique el docente antes de que termine el tiempo

Materiales permitidos

- Tus propios repositorios de las Semanas 6 a 9. Puedes copiar de ellos `lib/secureStorage.ts`, `lib/supabase.ts`, `context/AuthContext.tsx` y la configuración de Drizzle, Babel y Metro.
- Las guías del curso y la documentación oficial.
- Asistentes de IA, con la condición de registrar su uso en `AI-LOG.md`.

Reglas

- El trabajo es individual: no se comparte código ni proyecto de Supabase con otros estudiantes.
- Usa tu propio proyecto de Supabase. Si ya tienes uno del curso, puedes reutilizarlo creando la tabla nueva.
- En la app solo va la publishable key. Una secret key o el archivo `.env` en el repositorio anulan el criterio de entrega (ver evaluación).
- Lo que no esté en GitHub al terminar el tiempo no se califica.
- Si algo no funciona al final, entrega igual: se califica lo que sí funciona, y el README puede explicar lo que faltó.

Escenario y pantallas

Doña Rosa tiene una tienda en un barrio de Riohacha donde la señal se cae a menudo. Necesita una app para llevar el inventario: anotar qué productos tiene, cuántas unidades y a qué precio. La app debe funcionar siempre, con o sin internet, y guardar una copia en la nube para no perder nada si el celular se daña.

Cada producto tiene tres datos: **nombre**, **cantidad** (unidades en existencia) y **precio** (en pesos, sin decimales).

Pantallas obligatorias

Pantalla	Ruta sugerida	Qué debe tener
Inicio de sesión	`app/login.tsx`	Correo, contraseña, botón Ingresar, enlace a Registro y mensaje de error visible si falla
Registro	`app/registro.tsx`	Correo, contraseña, confirmar contraseña, validación de que coinciden y de al menos 6 caracteres, y enlace para volver al login
Principal (inventario)	`app/index.tsx`	Lista de productos, total de unidades y valor total del inventario, cantidad de productos pendientes de subir, botón Sincronizar, botón para agregar y botón Cerrar sesión
Formulario de producto	`app/producto.tsx`	Crear y editar (si recibe `id`), con validación: nombre obligatorio, cantidad y precio enteros mayores o iguales a 0

Comportamiento esperado

- Sin sesión solo existen Login y Registro. Con sesión solo existen Principal y Formulario.
- La sesión sigue iniciada al cerrar y volver a abrir la app.
- Crear, editar y eliminar productos funciona en modo avión.
- Al eliminar se pide confirmación (por ejemplo, manteniendo presionado el producto).
- Cada producto de la lista muestra un indicador de si ya está en la nube o está pendiente.
- Al tocar Sincronizar se suben los pendientes y se bajan los productos que el usuario tenga en la nube y no estén en el teléfono.

El diseño visual es libre. Se valora que sea claro, no que sea elaborado.

Requisitos técnicos

SQLite es la fuente de verdad: la app siempre lee y escribe en local. Supabase es el respaldo y se usa solo para iniciar sesión y al sincronizar.

1. Autenticación

- Supabase Auth con correo y contraseña (`signUp`, `signInWithPassword`, `signOut`).
- Sesión guardada en SecureStore (puedes reutilizar el adaptador por fragmentos de la Semana 8). No se acepta AsyncStorage.
- Rutas protegidas con `Stack.Protected`.
- Credenciales en `.env` con `EXPO_PUBLIC_SUPABASE_URL` y `EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY`. El repositorio lleva `.env.example`, nunca `.env`.

Antes de empezar, desactiva la confirmación por correo en la configuración del proveedor Email de Authentication; si no, el registro no inicia sesión.

2. Base de datos local (SQLite + Drizzle)

Tabla `products` con estas columnas:

Columna (TypeScript)	Tipo	Para qué
`id`	entero autoincremental	Llave local
`clientId`	texto único (UUID)	Identifica el mismo producto en el teléfono y en la nube
`name`	texto obligatorio	Nombre del producto
`quantity`	entero obligatorio	Unidades en existencia
`price`	entero obligatorio	Precio en pesos
`updatedAt`	`timestamp_ms`	Última modificación
`syncedAt`	`timestamp_ms`, opcional	Versión que quedó en la nube

- La migración se genera con `npx drizzle-kit generate` y se aplica con `useMigrations`.
- La lista de la pantalla principal es reactiva (`useLiveQuery`).
- Las pantallas no importan Drizzle directamente: usa un repositorio (`db/repositories/products.ts`) y un hook.
- Un producto está pendiente si `syncedAt` está vacío o es anterior a `updatedAt`. Cada creación o edición escribe `updatedAt: new Date()`.

> *No importes `expo-crypto` ni ningún paquete de Expo en `db/schema.ts`: drizzle-kit lee ese archivo con Node y falla con `Unexpected "typeof"`. Genera el UUID en el repositorio con `Crypto.randomUUID()`.*

3. Base de datos en la nube (Supabase)

Ejecuta este SQL en el SQL Editor de tu proyecto:

```
create table public.products (
  id bigint generated always as identity primary key,
  user_id uuid not null default auth.uid() references auth.users (id) on delete
cascade,
  client_id uuid not null unique,
  name text not null,
  quantity integer not null check (quantity >= 0),
  price integer not null check (price >= 0),
  updated_at timestamptz not null,
  created_at timestamptz not null default now()
);

alter table public.products enable row level security;

-- Cada usuario solo ve, crea, edita y borra sus propios productos
create policy "ver los míos" on public.products
  for select to authenticated using ((select auth.uid()) = user_id);
```

```
create policy "crear los míos" on public.products
  for insert to authenticated with check ((select auth.uid()) = user_id);

create policy "editar los míos" on public.products
  for update to authenticated
  using ((select auth.uid()) = user_id)
  with check ((select auth.uid()) = user_id);

create policy "borrar los míos" on public.products
  for delete to authenticated using ((select auth.uid()) = user_id);
```

4. Sincronización (botón Sincronizar)

En `services/sync.ts`, una función `syncProducts(userId)` que haga dos pasos, en este orden:

1. **Bajar.** Traer de Supabase los productos del usuario. Insertar en SQLite los que no existan localmente (comparando por `client_id`), marcándolos como sincronizados.
2. **Subir.** Enviar los productos pendientes con `upsert` por `client_id` y marcarlos como sincronizados con `syncedAt` igual al `updatedAt` que se subió.

```
// Fragmentos de apoyo
const { data, error } = await supabase.from('products').select('*');

await supabase.from('products').upsert(
  {
    client_id: p.clientId,
    name: p.name,
    quantity: p.quantity,
    price: p.price,
    updated_at: p.updatedAt.toISOString(),
  },
  { onConflict: 'client_id' }
);
```

La pantalla principal muestra un mensaje con el resultado (por ejemplo, "2 subidos, 1 bajado") o el error si no hay conexión.

Fuera de alcance

No se piden borrados sincronizados, resolución de conflictos, TanStack Query ni sincronización automática. Si te sobra tiempo, puedes agregarlos como puntos extra (ver evaluación).

Plan sugerido para los 90 minutos

La clave es tener algo funcionando y subido temprano, y mejorarlo después. Haz un commit y un push al final de cada bloque.

Minutos	Bloque	Al terminar debes tener
0 – 10	Preparación	Proyecto creado, SQL ejecutado en Supabase, `.env` y `.gitignore`, repositorio en GitHub con el primer push
10 – 30	Autenticación	Login, Registro y rutas protegidas funcionando

Minutos	Bloque	Al terminar debes tener
30 – 55	Base local	Esquema, migración, repositorio, pantalla principal y formulario con CRUD en SQLite
55 – 75	Sincronización	Botón Sincronizar que sube pendientes y baja lo que falta
75 – 90	Pruebas y entrega	Capturas tomadas, README completo y push final

No dejes la entrega para el último minuto: un `git push` que falla por credenciales puede tomar varios minutos en resolverse.

Entrega

Repositorio en GitHub

- Repositorio público llamado `parcial-mi-tienda-<apellido>`, por ejemplo `parcial-mi-tienda-gonzalez`.
- Al menos 4 commits con mensajes descriptivos, uno por bloque del plan. Un solo commit al final resta puntos.
- `.env` fuera del repositorio y `.env.example` incluido. Verifícalo antes del primer push con `git status`.
- La carpeta `drizzle/` con la migración sí va en el repositorio.

```
git init
git add .
git commit -m "Proyecto base y configuración"
git branch -M main
git remote add origin https://github.com/<usuario>/parcial-mi-tienda-<apellido>.git
git push -u origin main
```

Capturas de pantalla

Guarda las capturas en una carpeta `capturas/` dentro del repositorio, con estos nombres:

Archivo	Qué debe mostrar
`01-login.png`	Pantalla de inicio de sesión
`02-registro.png`	Pantalla de registro con un mensaje de validación visible (por ejemplo, contraseñas que no coinciden)
`03-principal.png`	Pantalla principal con al menos 3 productos, los totales y los indicadores de sincronización
`04-formulario.png`	Formulario editando un producto existente
`05-sin-conexion.png`	Un producto creado en modo avión, marcado como pendiente
`06-sincronizado.png`	La pantalla principal después de sincronizar, con el mensaje de resultado

Archivo	Qué debe mostrar
`07-supabase.png`	El panel de Supabase con la tabla `products` mostrando tus filas

Para capturar en el teléfono usa la combinación de botones de tu equipo y pásalas al computador por cable, correo o la nube que prefieras. Si el tiempo apremia, una foto clara de la pantalla también se acepta.

README.md

Debe incluir:

1. Tu nombre completo y código.
2. Cómo ejecutar el proyecto: crear `.env`, ejecutar el SQL, `npm install`, `npx expo start`.
3. Las siete capturas mostradas en el README, con `![Login](capturas/01-login.png)` y así con las demás.
4. Qué quedó funcionando y qué no, si algo faltó.

AI-LOG.md

Si usaste IA, registra al menos un prompt, qué corregiste del código que generó y, si la hubo, una alucinación detectada. Si no usaste IA, escríbelo en el archivo.

Antes de enviar el enlace, verifica

- ☐ El repositorio abre sin iniciar sesión en GitHub (es público).
- ☐ `.env` no aparece en GitHub.
- ☐ La carpeta `capturas/` tiene las siete imágenes y se ven en el README.
- ☐ El último commit tiene hora anterior al cierre del parcial.

Criterios de evaluación

La nota es sobre 5.0. Se califica lo que funciona en el repositorio entregado y lo que muestran las capturas.

Criterio	Peso	Completo	Parcial	No cumple
Autenticación	25%	Login, registro con validaciones y cierre de sesión; rutas protegidas; sesión en SecureStore que sobrevive al cierre	Login y registro funcionan, pero sin rutas protegidas o sin SecureStore	No se puede iniciar sesión
Base de datos local	25%	Esquema y migración con Drizzle, CRUD completo, lista reactiva y totales; funciona en modo avión	CRUD incompleto o sin totales	Datos en memoria o en AsyncStorage
Nube y sincronización	25%	Tabla con RLS, sube pendientes sin duplicar y baja lo que falta;	Solo sube, o duplica filas al	No hay comunicación

Criterio	Peso	Completo	Parcial	No cumple
		indicador de pendientes	repetir	con Supabase
Pantallas y validaciones	10%	Las cuatro pantallas, mensajes de error claros y confirmación al eliminar	Falta una pantalla o las validaciones	Varias pantallas faltantes
Entrega	15%	Repositorio público, 4 o más commits, README con las siete capturas, `.env.example` y AI-LOG	Faltan capturas o el README está incompleto	Sin repositorio o sin capturas

Penalizaciones

- `.env`, una secret key o la contraseña de la base en el repositorio: el criterio de Entrega queda en 0.
- Commits posteriores a la hora de cierre: no se tienen en cuenta.
- Código idéntico al de otro estudiante: el parcial se anula para ambos.

Puntos extra (hasta 0.5)

- Borrado sincronizado con borrado lógico (`deletedAt`).
- Regla de conflicto "gana el último cambio" al bajar productos que ya existen.
- Sincronización automática al recuperar la conexión.
- Búsqueda por nombre en la pantalla principal.

El docente puede pedirte que expliques brevemente una parte de tu código. Si no puedes explicarla, ese criterio no se califica como completo.